Interpreter audit and stress test

Bugs, wrong results, freezes, rendering problems, performance and missing features in the Regneark, with fixes and a stress-test suite.

Date	2 November 2026
Code audited	Commit `eb7763c` on `main`, plus your uncommitted change to `regneark.html` (the section feature)
Scope	`src/interpreter.py`, `parser.py`, `quantity.py`, `utils.py`, the chemistry and biotek functions it calls, `Assets_math_live/regneark.html`, and the Regneark parts of `src/gui.py` (bridge, tabs, open/save, LaTeX conversion)
Environment	Windows 11, Python 3.13 from the project `.venv`, PySide6 6.11, headless (`QT_QPA_PLATFORM=offscreen`). Nothing was opened on your desktop.
Source changes	None. Only new files in `tests/`.

Summary

I found **54 issues**: 7 critical, 9 high, 24 medium, 13 low and 1 note about work in progress. Every one was reproduced. Inputs and measured results are given with each finding.

The engine itself is hard to crash. In 1 600 fuzzed lines `ev()` never let an exception escape. The serious problems are elsewhere: calculations that never end and can't be stopped, a plot zoom that freezes the page for good, a damaged .crbn file that stops Carbon at start-up, a calculus path that runs Python code from the sheet, and a group of silent wrong answers. Function results are rounded to two decimals mid-calculation, `2x` ignores the variable x, Loop runs once too often, and Afstem prints wrongly balanced equations for formulas with parentheses.

Not that I enjoyed reading 7 000 lines of your code or anything. But the fixes are mostly small, and the ten at the top of the list remove nearly all of the real-world pain.

Severity	Count	Meaning
Critical	7	Crash, permanent freeze, security hole, or wrong answers in everyday use
High	9	Silent failures, data loss risk, wrong answers in specific cases, severe slowdowns
Medium	24	Broken feature with a workaround, misleading output, noticeable slowness
Low	13	Rare edge cases, cosmetics, clean-up
Info	1	Notes on unfinished work

Fix these ten first

ID	What to do
C-01	Plot zoom freezes the page: bound the grid loop and clamp the zoom (about 10 lines of JavaScript).
S-01	Stop passing calculus lines straight to `parse_expr`: allow-list tokens and names first.
C-02	Catch bad .crbn files in `load_regneark` and `setSheet`, and show a Danish message.
B-06	Remove the rounding inside `parser.atom()`. Round only for display.
B-01	Define `SYMPY_GLOBALS` (or add sqrt, Gamma and Erf to `SYMPY_FUNCTIONS`).
C-03 + C-04	Evaluate in a killable process, with a Stop action and a cap on Loop counts.
B-03	Use the existing `as_expr()` and a boundary that allows `2x`.
B-02	Compare with `"Loop"` in `exclude_loops`.
C-05	Catch formatting errors per value in `evaluate()`.
B-18	Use `split("\n")` instead of `splitlines()`.

1 Scope and method

- **Reading.** I read the Regneark code path end to end: LaTeX from MathLive, then `Latex_to_calc`, then `EV.ev`, then the formatting in `evaluate()`, then `showOutput` in the page. I also read open/save, tabs and the threading around it.
- **Probing.** 169 hand-written engine cases ran in a separate worker process with a time limit each, so a hang could be killed and recorded instead of blocking everything. Cases ran at three levels: calculator syntax (`EV().ev`), LaTeX through the real bridge (`_InterpreterBridge.evaluate`), and direct Python checks.
- **Editor.** `regneark.html` was loaded in a headless `QWebEngineView`, the same component the app uses, and driven with its own functions (`setSheet`, `showOutput`, typing commands, paste and wheel events). There were about 40 editor probes, and each heavy one got a fresh page.
- **Stress.** Sizes went far past normal use (200 000-term lines, 20 000-line sheets, 10^8 loop iterations, 5 000 variables, 1 000-function plots, deep zoom) to find where things break, freeze or slow down.
- **Fuzzing.** Seeded random sheets were run through the engine (4 × 400 lines) and through the LaTeX path (3 × 300 lines).
- **Your test suite.** I ran the existing suite twice and sorted every failure into intended change versus real problem.

Line numbers refer to the working tree on 2 November 2026. You were committing while I worked, so they may drift by a few lines.

2 Findings

IDs: **C** crash, freeze or hang · **S** security · **B** bug or wrong result · **R** rendering · **P** performance · **M** missing or incomplete feature · **E** error messages · **Q** code quality · **WIP** your unfinished section feature. Test names in `tests/test_stress_*.py` carry the same IDs.

Critical (7)

CRITICAL C-01 Zooming into a plot freezes the sheet for good

Where	`Assets_math_live/regneark.html:787` and `:793` (grid loops in `draw()` inside `plotElement`)
What happens	Ctrl+scroll into a `PlotFunktion` graph. After about 200 wheel steps the tab stops responding and never recovers (probe: still no answer 20 s later). Nothing in that tab works any more, including saving it.
Why	When the visible range is tiny compared with `x0`, `i = Math.ceil(view.x0 / sx)` is larger than 2^53. Then `i++` no longer changes `i`, so `i * sx <= view.x1` stays true forever.
Fix	Clamp the zoom (stop zooming when `view.x1 - view.x0 < 1e-9 * Math.max(1, Math.abs(view.x0))`) and loop over a bounded count instead of a float condition.

```
const first = Math.ceil(view.x0 / sx);
for (let k = 0; k < 200; k++) {
    const x = (first + k) * sx;
    if (x > view.x1) break;
    ctx.beginPath(); ctx.moveTo(px(x), 0); ctx.lineTo(px(x), h); ctx.stroke();
    ctx.fillText(tidy(x, 12), px(x) + 3, labelY);
}
```

CRITICAL C-02 A damaged .crbn file stops Carbon at start-up and half-loads in the editor

Where	`src/gui.py:3018` (`load_regneark`), `src/gui.py:3076` (start-up call), `regneark.html:933` (`setSheet`)
What happens	Empty, binary, non-UTF-8 and `{}` files raise `JSONDecodeError`, `UnicodeDecodeError` or `KeyError` (all four confirmed). When the file is opened by double-clicking, this happens in `__main__` before the window is shown, so Carbon never starts. The windowed build (`console=False`) shows PyInstaller's "Unhandled exception" box with a Python traceback instead of a Danish message. Inside the editor, one bad plot row makes `setSheet` throw: the lines appear but no results, and the old undo history survives. A file whose rows aren't a list wipes the previous sheet except its first line.
Fix	Wrap the read in `try/except (OSError, UnicodeDecodeError, json.JSONDecodeError, KeyError, TypeError)` and show a Danish `QMessageBox` ("Filen kunne ikke åbnes"). Check that `rows` is a list of `[kind, text]` pairs with a known kind before calling `setSheet`. In `setSheet`/`showOutput`, put each row in `try/catch` and skip bad rows.

CRITICAL C-03 Runaway calculations can't be stopped

Where	`src/interpreter.py:283` (Loop), `:321` (Løs), `:912` (calculus); `src/gui.py:748-752` (run)
What happens	`Loop(2,2,100000000)` never finishes (killed after 20 s; 100 000 iterations take 2 s, so about 33 min, plus 10^8 result rows). `summation(1/k, (k, 1, 100000))` was killed after 60 s. `Løs(x^9 + 3x^7 - 5x^3 + 11 = Sin(x)*Exp(x))` takes 23 s. There is no feedback and no way to cancel. Pressing Ctrl+Enter again starts another thread while the old one keeps running.
Fix	Run evaluation in a separate process (a `multiprocessing` worker or `QProcess`) that can be killed. Add a "Stop" action (for example Esc while running), cap Loop counts (such as 10 000) and the number of result rows, and give symbolic calls a time limit.

CRITICAL C-04 A SystemLøs time-out leaves solver threads running and makes the whole app about 200x slower

Where	`src/interpreter.py:412-421`
What happens	When `sympy.solve` takes more than 20 s, SystemLøs falls back to `nsolve`, but the `ThreadPool` keeps computing in the background. Measured with a fixed hash seed (0-5 all behave the same): 4 extra threads still alive afterwards, and `BjerrumDiagram` with 12 values then took 53 s, 49 s and 44 s instead of 0.24 s. All tabs share this Python process, so every later calculation crawls until the abandoned solve finishes.
Why	Python threads can't be killed, so the time-out only stops waiting for the thread. It doesn't stop the thread.
Fix	Use a process for the time limit, for example `multiprocessing.Pool(1)` with `pool.terminate()` on time-out, or the process-based evaluation from C-03.

CRITICAL S-01 Calculus lines are run as Python code

Where	`src/interpreter.py:912-913`
What happens	Any line containing `integrate`, `diff`, `limit` or `summation` goes straight into sympy's `parse_expr`, which evaluates the text with Python's `eval`. A harmless probe, `integrate(x, x) + len('abcd')`, returned $x^2/2 + 4$, so Python's built-in functions are reachable. Carbon is now registered as the program for .crbn files, which are meant to be shared, so a sheet from someone else could run any Python on your computer when you press Ctrl+Enter.
Fix	Never pass raw user text to `parse_expr`. First check the tokens: allow numbers, operators, parentheses, single-letter symbols and an explicit list of function names, and reject underscores, dots, quotes and square brackets. Then pass a `global_dict` that contains only what you allow, plus `Symbol`, `Integer`, `Float` and `Rational`. Use the same check in `prep_sympy`, where it currently only works by accident because of B-01.

CRITICAL B-01 Square roots break Løs, Simplificer, PlotFunktion and the other symbolic functions

Where	`src/interpreter.py:958` (`prep_sympy`)
What happens	`SYMPY_GLOBALS` is not defined anywhere. Any function name that isn't in `SYMPY_FUNCTIONS` (sqrt, Round, Gamma, Erf) makes `prep_sympy` raise `NameError`, shown as "Error: name 'SYMPY_GLOBALS' is not defined". The help example `Simplificer(sqrt(12) + sqrt(27))` fails, and because a typed √ becomes `sqrt(...)`, every Løs/PlotFunktion/Regression/NumeriskLøs/SystemLøs with a square root fails.
Fix	Define the list, for example `SYMPY_GLOBALS = {"sqrt": sympy.sqrt, "Gamma": sympy.gamma, "Erf": sympy.erf}`, and pass it in `local_dict` together with `SYMPY_FUNCTIONS`.

CRITICAL B-06 Function results are rounded to 2 decimals in the middle of a calculation

Where	`src/parser.py:174`
What happens	Every function result is rounded to `config.rounding_decimals` before it is used further. Probes: `Exp(-10)` gives 0, the Arrhenius term `Exp(-50000/(8.314*298))` gives 0, `Sin(1)*1000` gives 20 (should be 17.45), `Ln(2)*100` gives 69 (69.31), `Log(1.001)` gives 0, and `Round(3.14159, 3)` gives 3.14.
Fix	Don't round in `atom()`. Round only when the final result is displayed.

High (9)

HIGH C-05 One unprintable result hides every result in the sheet

Where `src/gui.py:709-745` (formatting loop in `evaluate`, outside the `try`)

What happens If turning a value into LaTeX raises, for example an exact integer with more than 4 300 digits from `integrate(x,x)+10^5000`, `evaluate()` raises inside the worker thread. The `finished` signal never fires, so the sheet shows no results and no error.

Fix Wrap the formatting of each value in `try/except` and emit an error row for that line only.

HIGH C-06 A sheet containing the word "export" is calculated on the GUI thread

Where `src/gui.py:749-751`

What happens `'export' in data.lower()` looks at the whole sheet, comments included. The entire sheet is then evaluated on the GUI thread (probe: `run()` blocked for the whole evaluation), so one slow line, such as a 20 s SystemLøs, freezes the window.

Fix Evaluate in the worker as usual and only open the Bjerrum export window on the GUI thread, for example by sending the pKs values back with a signal.

HIGH B-02 Loop runs its lines one time too many

Where `src/interpreter.py:221` (`exclude_loops`)

What happens `x = 0`, `x += 1`, `Loop(2,2,3)` ends with x = 4 instead of 3. With the Loop above its lines, they also run once more after the loop.

Why `call.group(1) == "loop"` (lower case) is never true, so the lines inside loops are never skipped in the normal pass.

Fix Compare with `"Loop"` and drop the inner `if func == "Loop"`.

HIGH B-03 Variables are pasted in as text, so many formulas give wrong answers

Where `src/interpreter.py:942-953` (`prep_ev`). The fix, `as_expr()` at `:29`, already exists but is never called

What happens `x = 5`, then 2x gives "2.0 x": x is treated as a unit, because `\b` doesn't match between 2 and x. `a = -2`, then `3 a` gives 1, because it becomes 3 - 2. `a^2` gives -4. `a = 3[m]`, then `a^2` gives 3 m². `v = 2[m/s]`, then `10/v` gives 5 1/(m·s) instead of 5 s/m. Some of these are already an xfail in `test_interpreter.py`.

Fix Match names with `(?<![A-Za-zæøåÆØÅ_])name(?!\w)`, so a digit may come before the name (2x) but not after it, and insert the value in parentheses, which is what `as_expr()` does.

HIGH B-05 Units of the same kind can't be added

Where `src/quantity.py:140-153`

What happens `1[km] + 500[m]` and `1[kg] - 200[g]` give "Error: Error cant add diffrent units".

Fix When the units differ, convert both sides to SI. If the dimensions match, add the SI values.

HIGH B-17 Afstem and Analyse ignore parentheses and print wrong answers without warning

Where `src/chem.py:249-255` (`reaction_parser`), `:380-399` (`molecule_formula`)

What happens `Afstem(K4Fe(CN)6 + KMnO4 + H2SO4 -> KHSO4 + Fe2(SO4)3 + MnSO4 + HNO3 + CO2 + H2O)` prints "10K4Fe(CN)6 + 28KMnO4 + 101H2SO4 -> 68KHSO4 + ...", which is wrong (the correct coefficients are 10, 122, 299, 162, 5, 122, 60, 60, 188). `Analyse(Ca(OH)2)` gives the empirical formula CaOH and a molar mass of 57.09 (should be 74.09). Your tests already mark the Ca(OH)2 cases xfail. It's High because the answer looks right and isn't.

Fix Expand `(...)n` groups recursively before counting atoms, and check that the coefficients really balance every element before printing.

HIGH B-18 A line-separator character moves all later results to the wrong lines

Where `src/interpreter.py:255` (`s.splitlines()`)

What happens `splitlines()` also splits at U+2028, U+0085, form feed and similar characters, which can come from text pasted from the web or Word. The GUI joins lines with \n and maps results back by index, so every later result is shown next to the wrong line (probe: line 5 showed 4.0 and line 7 showed 5.0).

Fix Use `s.split("\n")`.

HIGH M-04 Closing and saving: the new close prompt can lock the window, and save errors are silent

Where	`src/gui.py:3041` (`closeEvent`, added in commit eb7763c during this audit), `src/gui.py:2946-2954` (`save_regneark`)
What happens	The new close prompt asks every tab through `runJavaScript` and only closes once all of them have answered. A tab that is frozen (C-01) or busy (P-03) never answers, so Carbon can't be closed with the X button. The prompt also appears right after saving. Saving writes the file inside a JavaScript callback, so a failed write (no permission, full disk, file open elsewhere) raises in the callback and nothing is shown, even though the tab was already renamed.
Fix	Show the prompt anyway after a short `QTimer` (1-2 s). Keep a per-tab "changed" flag that is set on input and cleared on save. Catch `OSError` in save and show a message, and write to a temporary file that is then renamed.

HIGH P-03 Big sheets and long lines make the editor sluggish

Where	`regneark.html:933-949` (`setSheet`), `:167-171` (`recolour`), plus MathLive re-rendering the whole line
What happens	Opening a file: 200 lines take 0.8 s, 1 000 lines take 10 s (plus 6 s before the page answers again) and 3 000 lines take 79 s. Result rows: 500 take 2.9 s, 1 000 take 9.7 s (+5 s) and 2 000 take 34 s (+21 s), so `Loop(1,1,2000)` freezes the tab for almost a minute and `Loop(1,1,20000)` for roughly an hour. Typing, with the real input handlers: 7 ms per key on a short line, 305 ms on a 1 000-term line and 2.6 s on a 3 000-term line.
Fix	Build the lines in a detached container and insert them in one go. Create math-fields only for visible lines, or render off-screen lines as static rows. Recolour only the changed part, or debounce `recolour` by about 150 ms.

Medium (24)

MEDIUM B-04 Anything before "=" becomes a variable, even a number

Where	`src/interpreter.py:869-910`
What happens	`2 = 5`, then 2+2 gives 10. `2 <= 3` creates a variable called "2 <" and prints 3.0.
Fix	Require `re.fullmatch(r"[^\W\d]\w*", name)` and refuse names of functions and constants.

MEDIUM B-07 Help() refuses the maths functions it has help text for

Where	`src/interpreter.py:835`
What happens	`Help(Ln)`, `Help(Sin)` and `Help(Round)` give "Ukendt function ...". `help_text` has those entries, but the check uses `error_text`.
Fix	Check `input not in self.help_text`, and list the maths functions in `Help()`.

MEDIUM B-08 SystemLøs without Variabler= shows a raw Python error

Where	`src/interpreter.py:405`
What happens	"cannot access local variable 'variables' where it is not associated with a value".
Fix	Set `variables = None` first and raise `error_text["SystemLøs"]` if it's still None.

MEDIUM B-09 MængdeBeregning breaks with spaces or variables

Where	`src/interpreter.py:757-768`, `:1025-1037`
What happens	`MængdeBeregning(m = 10[g], M = 58.44[g/mol])` gives "Ukendt input 'm '". `m=masse` gives "could not convert string to float: 'masse'".
Fix	Strip the key, and read values with `input_prep()` instead of `float()`.

MEDIUM B-10 Cubic metres and litres are different dimensions

Where	`Assets/units.csv` (l is its own base unit), `src/quantity.py:199-218`
What happens	`EnhedKonvertering(1[m^3], l)` gives "cannot convert {'m': 3} to l". cm³ to ml fails too (already an xfail). The other direction, `1000[l]` to m^3, works.
Fix	Define l as 0.001 m³ in the table (dimension `{"m": 3}`) so all volumes share one dimension.

MEDIUM B-11 Unknown target units are ignored, and common units are missing

Where	`src/quantity.py:247-269`, `Assets/units.csv`
What happens	`EnhedKonvertering(5[km], mxyz)` gives 5000 m with no error. `km/t` and `km/h` fail ("Ukendt enhed: t"), even though the unit help text uses 10[km/t] as its example. °C is unknown, and `m/s/s` fails.
Fix	Raise on any unknown part. Add t/h, °C (with offset handling), bar, atm and similar units. Add up repeated units (already an xfail).

MEDIUM B-12 The percent functions garble compound units

Where	`src/chem.py:615-623`
What happens	`MasseProcent(2[g/l], 4[g/l])` gives "50 1/l^2%".
Fix	Check the units, then compute `top.value / bottom.value * 100` directly instead of re-parsing a string.

MEDIUM B-13 BjerrumDiagram doesn't find acid names after a space

Where	`src/interpreter.py:659`
What happens	`BjerrumDiagram(ethansyre, carbonsyre)` gives "Ukendt enhed: carbonsyre".
Fix	`part = part.strip()` before the lookup.

MEDIUM B-14 Analyse accepts nonsense

Where	`src/chem.py:402-469`
What happens	`Analyse(co2)` prints "Molar masse: 0" and an empty formula instead of an error.
Fix	Raise if no elements are found, or if anything other than element symbols, digits and parentheses remains.

MEDIUM B-15 Pasting several lines turns % and \ into LaTeX

Where	`regneark.html:477` (`onPaste` passes raw text to `newLine`)
What happens	Pasting "50% af 20" shows only "50", because the rest became a LaTeX comment, and the line then fails when run. "a\b" shows "a".
Fix	Escape plain text before `setLine` (% to \%, # to \#, \ to \backslash), or insert each line with `mf.insert(text, {format: "ascii-math"})`.

MEDIUM B-16 A Loop typed with spaces points at the wrong lines after the first run

Where	`regneark.html:326` (`LOOP` regex)
What happens	`Loop(2, 2, 3)` with spaces is not tracked. After Ctrl+Enter the result rows shift the numbering, but the text still says (2, 2, 3), which now points at a result row, so the next run loops over nothing.
Fix	Allow optional spaces and `\:` between the numbers in the regex.

MEDIUM B-19 Round() rounds halves to even

Where	`src/parser.py:38`
What happens	`Round(2.5)` gives 2 and `Round(0.125, 2)` gives 0.12. Danish school maths rounds halves up. There is a similar xfail for `significant_figures`.
Fix	Use `Decimal(str(x)).quantize(..., rounding=ROUND_HALF_UP)`.

MEDIUM B-20 The non-linear regressions start from guesses of all ones

Where	`src/interpreter.py:536-564`, `src/utils.py:184`
What happens	`HarmReg` on clean data from 2·sin(3x+0.5)+1 (40 points) returns 0.214·sin(0.869x+1.811)+1.038 with R^2 = 0.0105. The help text calls the parameters A, ω, φ, k, but the output uses a, b, c, d.
Fix	Pass `p0` to `curve_fit`. For HarmReg: offset = mean(y), amplitude = (max-min)/2, ω from the FFT peak. For LogistReg: c = max(y). For ExpReg: fit log(y) linearly first.

MEDIUM B-21 Roots of negative numbers give complex-number garbage

Where	`src/parser.py:133`, `:168-169`
What happens	`sqrt(-4)` shows "(1.22465·10^-16+2j)". `(-8)^(1/3)` gives (1.0000000000000002+1.7320508075688772j). The power case is already an xfail.
Fix	Raise a Danish error when the base is negative and the exponent isn't a whole number, or return the real root for odd roots.

MEDIUM R-01 Result rows can be edited, and the edits are saved

Where	`regneark.html:865-870` (`showOutput`)
What happens	Clicking the result of `1+1` and typing "999" turned "2.0" into "2.0999", and `getSheet` saved the changed value.
Fix	Set `el.readOnly = true` on output fields. Selecting and copying still work.

MEDIUM R-05 Ion charges are written as double superscripts

Where	`src/gui.py:637-660` (`calc_to_latex`)
What happens	A redox result from Afstem contains `Fe^{2}^{+}`, which is invalid LaTeX. This is already an xfail.
Fix	Handle `\^(\d*[+-])` before the generic superscript rule, giving `^{2+}`.

MEDIUM P-01 calc_to_latex takes quadratic time on long words

Where	`src/gui.py:648` (the fraction regex)
What happens	2 000 characters take 0.03 s, 4 000 take 0.12 s, 8 000 take 0.47 s and 16 000 take 1.94 s. A string variable holding a 20 000-letter DNA sequence takes 3 s just to display.
Why	`([^\s/]+)/` matches the whole word and then backtracks one character at a time, from every start position.
Fix	Only run the fraction rule on words that contain "/" (split on whitespace first), or use `(\S+?)/(\S+)`.

MEDIUM P-02 Every assignment rebuilds the variable regex

Where	`src/interpreter.py:908`, `:930-940`
What happens	2 000 variables take 13 s and 5 000 take 98 s. The cost grows with the square of the number of variables.
Fix	Substitute with one generic identifier pattern and a dictionary lookup, instead of an alternation of every name that is rebuilt after each assignment.

MEDIUM P-04 The undo history grows without limit

Where	`regneark.html:545-549`
What happens	300 keystrokes on a 500-line sheet stored 300 snapshots, 8.9 MB of undo history (about 30 KB per key, a full copy of every line). Each keystroke took 33 ms, because the whole sheet is serialised twice. Nothing is ever removed, so a long session on a big sheet keeps growing.
Fix	Cap `undoStack` (for example at 200 entries) and store only the lines that changed.

MEDIUM M-01 π and e aren't numbers

Where	`src/parser.py` (constants are read as units)
What happens	`2π` gives "2.0 π", `Sin(π)` gives an error and `e^2` gives "1.0 e^2". Only `Exp(1)` works.
Fix	Treat π and e as constants (`math.pi`, `math.e`) unless the user has defined a variable with that name.

MEDIUM M-02 Function results can't be stored or used further

Where	`src/interpreter.py:275-276` (a call must be the whole line)
What happens	`x = pH(4.76, 0.1)` gives 'Ukendt funktion "pH"'. The special functions only work as a line of their own.
Fix	Let the parser call the special functions and return a quantity, or handle `name = Func(...)` by evaluating the call and storing its result.

MEDIUM M-03 Common LaTeX from MathLive isn't translated

Where	`src/gui.py:573-634` (`Latex_to_calc`)
What happens	`\ln(2)` gives 'Ukendt funktion "ln"' because it is mapped to lower-case ln. `6\div3`, `\sqrt[3]{8}`, `25[°C]` (written as `^{\circ}`) and `\le` all fail.
Fix	Map \ln to Ln, \log to Log, \exp to Exp, \div to /, \sqrt[n]{x} to x^(1/n) and ^{\circ} to °.

MEDIUM E-01 Python's English error texts reach the user

Where	the evaluation boundary in `src/interpreter.py:922-924`
What happens	Messages users see include "float division by zero", "math domain error", "math range error", "(34, 'Result too large')", "invalid literal for int() with base 10: '2.5'", "Could not find root within given tolerance...", "Element not found: Xx", "too many values to unpack" and "operands could not be broadcast together...".
Fix	Catch `ZeroDivisionError`, `OverflowError` and the common `ValueErrors` where `ev()` turns exceptions into text, and translate them into Danish.

MEDIUM E-03 Misleading errors

Where	`src/parser.py:179-180` (unknown names become units)
What happens	An undefined variable gives "cant add diffrent units" (`y + 1`) or is silently treated as a unit (2y gives "2.0 y"). Using a list variable gives "Brug punktum som decimal tal ikke komma", and a text variable is shown as a unit ("1.0 hej").
Fix	When a name is neither a variable nor a known unit, say "Ukendt variabel eller enhed: y".

Low (13)

LOW R-02 A plot with very many functions throws instead of drawing

Where	`regneark.html:745` (`Math.min(...ys)`)
What happens	With 300 functions the plot draws in 0.4 s. With 1 000 functions `showOutput` throws `RangeError: Maximum call stack size exceeded`, and every result row after the plot is left out.
Fix	Find the minimum and maximum with a plain loop instead of spreading the array into function arguments.

LOW R-09 Very long results sometimes make MathLive throw, and the rows after them disappear

Where	`regneark.html:849-873` (`showOutput` has no `try/catch` around `el.value = latex`)
What happens	A 210 000-character result (30 000 "Met, " tokens, for example from a long DNATilPeptid) rendered in 0.16 s on fresh pages every time. Inside the pytest process the same call failed in four of five runs: once caught as a `RangeError`, and three times as an empty result, which is what QtWebEngine returns when a script throws. When it throws, `showOutput` stops, so every result after that row is missing.
Fix	Put a `try/catch` around each row in `showOutput` and show an error row instead. Cut results longer than about 20 000 characters with a note such as "(forkortet)".

LOW R-06 Some results lose characters or produce broken LaTeX

Where	`src/gui.py:637-660`
What happens	`s = {1,2,3}` is displayed without its braces (`\mathrm{{1,2,3}}`). Text containing } or & produces broken LaTeX: `calc_to_latex('a}b')` gives `\mathrm{a}b}`.
Fix	Escape { } & # $ ~ ^ in plain text before wrapping it, the same way `latex_text` does.

LOW R-08 Plot ranges and degree defaults hide the curve

Where	`regneark.html:736-748`, help text in `src/interpreter.py`
What happens	The y-range uses the absolute min and max of all samples, so one steep part (such as `Exp(x^2)`) flattens the rest. Sin is in degrees, so the help example `PlotFunktion(x^2, Sin(x), Interval=(-5,5))` draws Sin as an almost flat line. With many Bjerrum values the legend covers the plot (matplotlib warns that tight_layout failed at 300 values).
Fix	Use percentiles (for example 2-98 %) for the automatic range, and change the help example to `sin(x)` or a 0-360 interval.

LOW P-05 Smaller optimisations

Where	several places
What happens	The whole sheet is recalculated on every Ctrl+Enter. `Løs` parses its input twice (`interpreter.py:327` and `:340`). `EV()` rebuilds its function, error and help dictionaries for every instance, and `help()` creates a new `EV`. The seven regression cases (`interpreter.py:460-564`) are copies of each other and could be one table of {name: formula}.
Fix	Make the dictionaries class constants, parse once, and replace the regression cases with one table.

LOW M-05 Smaller missing features

Where	`src/gui.py`
What happens	There is no Ctrl+S, and "Gem" always asks for a path. There is no way to stop a calculation, no autosave or recovery, and no comparisons (<, >). `Help()` doesn't list the maths functions.
Fix	Remember each tab's path and add Ctrl+S. The stop action comes with C-03.

LOW E-02 Spelling and language in messages

Where	`src/quantity.py:145,153`, `src/parser.py:158,182`, `src/interpreter.py:837`
What happens	"Error cant add diffrent units" (English, with spelling mistakes), "udtrykket sluter for tidligt" (slutter), "magler" (mangler), "Ukendt function ... hjælpe text" (funktion, hjælpetekst). Six failing parser tests already catch some of these.
Fix	Fix the texts. The failing tests then pass.

LOW B-22 Variables can overwrite function names

Where	`src/interpreter.py:869-910`
What happens	`Sin = 5`, then `Sin(30)` gives 150.
Fix	Refuse names of functions (see B-04).

LOW B-23 "#" inside a text value starts a comment

Where	`src/interpreter.py:263`
What happens	`s = 'a#b'` gives an error about quotes.
Fix	Only treat # as a comment when it is outside quotes.

LOW B-24 Loop edge cases

Where	`src/interpreter.py:283-319`
What happens	A count of 0 or less silently does nothing. `Loop(1,1,2.5)` shows a Python error. An end line past the end of the sheet silently stops early. Loops inside loops use absolute line numbers relative to the inner sheet.
Fix	Check the range and the count and give Danish errors.

LOW B-25 DNATilPeptid translates past stop codons

Where	`src/biotek.py:163-201`
What happens	`TAAATGGCC` gives "Stop, Met, Ala". A long invalid strand puts every invalid codon into the error message.
Fix	Stop at the first stop codon (or mark it), and cut the error message short.

LOW S-02 Image rows in .crbn files aren't checked

Where	`regneark.html:858`
What happens	An image row containing `https://...` is used as `img.src` as it is.
Fix	Only accept `` values.

LOW Q-01 Dead code and leftovers

Where	`src/interpreter.py`
What happens	`exclude_loops` (broken, see B-02), `equation_simlify` (unused, missing `self`), `as_expr` (unused, but it is the fix for B-03), `output_unit` (unused). Unused imports: `Figure`, `Path`, `pKb_values`. `var_type` is set to both "Quantity" and "quantity".
Fix	Remove them, or use them where they were meant to be used.

Info (1)

INFO WIP-01 Section feature (uncommitted): addSection can't be called yet

Where	`regneark.html:758` (inside `plotElement`), `src/gui.py` (`SegmentedControl.setCurrent`)
What happens	`addSection` is declared inside `plotElement`, so from Python `typeof addSection` is "undefined". Once it can be called, clicking into a section line reports the mode "section", and `SegmentedControl.setCurrent` raises `KeyError` because there is no such button. Sections loaded from a file come back as plain lines (no styling yet).
Fix	Move `addSection` to the top level of the script, and ignore unknown modes in `setCurrent`.

3 Stress-test results

OK means it worked within a reasonable time. **SLOW** means it worked but a user would notice. **FLAKY** means it sometimes failed. **FAIL** means a hang, freeze, crash, wrong output or unusable time. Engine cases ran in a fresh process each, and editor cases in a fresh headless page each.

Engine (Python)

Case	Size	Time	Verdict	Note
1+1+…+1	200 001 terms	0.72 s	OK	
One line of 1 MB	500 001 terms	1.65 s	OK	
Sheet of many lines	20 000 lines	0.23 s	OK	
Nested parentheses / power tower	1 000 / 2 000 levels	instant	OK	English message (E-01)
Loop	10 000 / 100 000 iterations	0.2 s / 2.0 s	OK	result rows: see editor
Loop inside Loop	10^6 iterations	20.1 s	SLOW	C-03
Loop	10^8 iterations	killed at 20 s	FAIL	C-03
Variables	2 000 / 5 000	13.4 s / 97.8 s	FAIL	P-02
Variable name	100 000 characters	0.15 s	OK	
Emoji, BOM, NUL, RTL marks	4 lines	instant	OK	Danish error
10^309, 9^9^9, 1e400-1e400		instant	OK	inf/nan shown, raw errno text (E-01)
Løs, hard polynomial + trig		22.8 s	SLOW	then "Kunne ikke løses"
integrate(Exp(x^2)·Sin(x^3))		8.3 s	SLOW	returned unevaluated
summation(1/k, k = 1..100 000)		killed at 60 s	FAIL	C-03
integrate(x,x) + 10^5000		instant	FAIL	whole sheet blank (C-05)
SystemLøs, hard system		20.7 s	FAIL	4 threads leak, app ~200× slower (C-04)
DNATilPeptid	300 000 bases	0.23 s	OK	
PlotFunktion	500 functions	1.0 s	OK	
BjerrumDiagram incl. PNG	12 / 80 / 300 values	0.27 / 1.05 / 3.8 s	OK	legend covers plot at 300
LinReg	50 000 points	0.51 s	OK	
Regression	12 parameters	0.14 s	OK	
calc_to_latex	16 000-character word	1.94 s	FAIL	quadratic (P-01)
Fuzzed sheets through ev()	1 600 lines	about 5 s per 400 lines	OK	no exception escaped
Fuzzed LaTeX through evaluate()	900 lines	under 4 s per 300 lines	OK	valid rows every time

Editor (regneark.html in QtWebEngine)

Case	Size	Time / result	Verdict	Note
Open a file	200 lines	0.8 s + 0.5 s busy	OK	
Open a file	1 000 lines	10.1 s + 6.3 s busy	FAIL	P-03
Open a file	3 000 lines	79 s	FAIL	P-03
Show result rows	500 / 1 000 / 2 000	2.9 / 9.7 / 33.9 s	FAIL	quadratic (P-03)
One huge result	210 000 characters	0.16 s, sometimes RangeError	FLAKY	R-09

Case	Size	Time / result	Verdict	Note
Typing	short line	7 ms per key	OK	
Typing	1 000 / 3 000 terms	305 ms / 2.6 s per key	FAIL	P-03
Undo history	300 keys, 500 lines	8.9 MB kept	FAIL	P-04
Plot	300 functions	0.42 s	OK	
Plot	1 000 functions	RangeError	FAIL	R-02
Ctrl+scroll zoom into a plot	≤ 200 wheel steps	page frozen for good	FAIL	C-01
Paste "50% af 20" + newline		shows "50"	FAIL	B-15
Damaged plot row in a file		throws, half loaded	FAIL	C-02

4 State of the existing test suite

Two full runs gave **54 failed, 702 passed, 22 xfailed**. On the first run the three `test_file_open` tests also errored, and on the second they passed.

Group	Tests	Verdict
`chem.pH` / `pOH` now take a float concentration, but the tests pass strings	31	Tests are out of date
`chem.procent` now takes quantities	4	Tests are out of date
`elemental_analasis` now gives mass percent (matches the help text)	1	Tests are out of date
`calc_to_latex` now turns a/b into fractions	2	Tests are out of date
Assignments now print their value	3	Tests are out of date
Text variables now need quotes	5	Tests are out of date
Loop and parser messages changed	2 + 4	Out of date, but they also catch the typos "sluter" and "magler" (E-02)
`test_unknown_function` now passes (XPASS strict)	1	Fixed: remove the xfail marker
`build.spec` doesn't bundle `Assets_Ketcher`	1	Real: the installed app has no molecule editor
`test_file_open`: 9 s subprocess time limit, but importing `gui` takes several seconds	3	Flaky

- **No hang protection on Windows.** The 10 s guard in `conftest.py` uses `signal.SIGALRM`, which doesn't exist on Windows, so one hanging test blocks the whole run. Use `pytest-timeout` (thread method), or run risky cases in a subprocess the way the new stress tests do.
- Your 17 existing xfail markers already document real bugs (parentheses in formulas, half-to-even rounding, cm³ to ml, charge superscripts, variable pasting and more). I list those as known and still open, not as new discoveries.

5 The new stress tests

Three new files in `tests/`, all headless and following your existing conventions. A bug that still exists is marked `xfail(strict=True)` with its ID in the reason. When you fix it, the test XPASSes and fails, which tells you to remove the marker. Anything that can hang runs in a subprocess with a time limit, so the suite keeps going.

File	What it covers
`tests/test_stress_interpreter.py`	Fuzzing (4 × 400 lines), extreme inputs, 200 000-term and 20 000-line sheets, 100 000 loop iterations, plus a regression test for each engine finding (hangs, thread leak, code execution, wrong results, messages).
`tests/test_stress_gui.py`	LaTeX fuzzing through the real bridge (3 × 300 lines), quadratic `calc_to_latex`, damaged files, the export threading, line splitting, formatting crashes and rendering details.
`tests/test_stress_regneark.py`	Editor: opening big files, result rows, a 210 000-character result, typing cost, undo growth, editable results, paste, loop renumbering, damaged rows, 300/1 000-function plots, remote images, the section WIP and the zoom freeze (in a killable subprocess).

Final run on 2 November 2026: **22 passed, 71 xfailed, 0 failed** in 3 min 47 s. The 71 xfails are the bugs in this report that still exist. The slowest tests are the deliberate time-outs: 45 s for the harmonic sum, 30 s for the huge Loop and 26 s for the SystemLøs leak.

Run them like this in PowerShell, headless and in the project venv:

```
$env:QT_QPA_PLATFORM = "offscreen"
.venv\Scripts\python.exe -m pytest -q -rxX `
    tests\test_stress_interpreter.py `
    tests\test_stress_gui.py `
    tests\test_stress_regneark.py
```

6 What already works well

- `ev()` turns every Python exception into an error row. Fuzzing never got one past it, and recursion-depth errors are caught too.
- Plot JavaScript loaded from files is checked against `SAFE_JS`, and all 27 sympy outputs I tried pass it, so valid plots survive saving and reopening. Results reach the page through MathLive and `textContent`, never `innerHTML`.
- Cases that matter in practice are fast: 20 000 lines in 0.23 s, a 1 MB line in 1.7 s, LinReg on 50 000 points in 0.5 s and 100 000 loop iterations in 2 s.
- Afstem handles big plain formulas correctly (for example 2C57H110O6 + 163O2 -> 114CO2 + 110H2O) and refuses reactions without a unique answer.
- Loops written without spaces are renumbered correctly when result rows push lines down.

7 Suggested order of work

- **First (small changes, big effect):** C-01, C-02, S-01, B-06, B-01, B-02, B-18, C-05, and the time-out for the new close prompt (M-04).
- **Then the architecture fix:** run evaluation in a killable worker process with a Stop action and a Loop cap (C-03, C-04, C-06). This also stops a slow sheet from slowing down every other tab.
- **Then correctness:** B-03 (use `as_expr`), B-05, B-17, B-04, B-10, B-11, B-12, B-19, B-20, B-21, M-01.
- **Then the editor:** P-03 (build lines in one go, lazy math-fields), P-04 (cap undo), R-01 (read-only results), B-15, B-16.
- **Finally:** messages (E-01 to E-03), clean-up (Q-01, P-05), and updating the 45 out-of-date tests so the suite is green again.

That's everything. D-don't get used to me doing this much for you.